ibm-saas-platform /
sols-ai-chat-design-doc

Code Issues Pull requests 2 Projects Wiki Security Insig

sols-ai-chat-design-doc session management session gateway
high-level-arch.md

MANSURAH Create high level arch md 5d01 .ce · yesterday

236 lines (153 loc) · 6.41 KB

sols-ai-chat-design-doc session management session gateway high-level-arch.md Top

Preview Code Blame Raw

title	author	date	version	status	reviewers	cor
Enterprise Session Management Design for Solis	Mansura Habiba	2026 01 22	1 0	Draft	Carlos	Inte

High level Archiecture for Session Gateway

Target Outcome

We want a session system where an auditor can ask

- Which user started this session under which tenant in which region?
- What data was stored vs ephemeral and for how long?
- Which tools were called with what scopes and was data redacted?
- Did any concurrent actions conflict and how were conflicts handled?
- Delete Identify session per user per teanant

Architecture pattern

Add a Session Gateway in front of Langflow

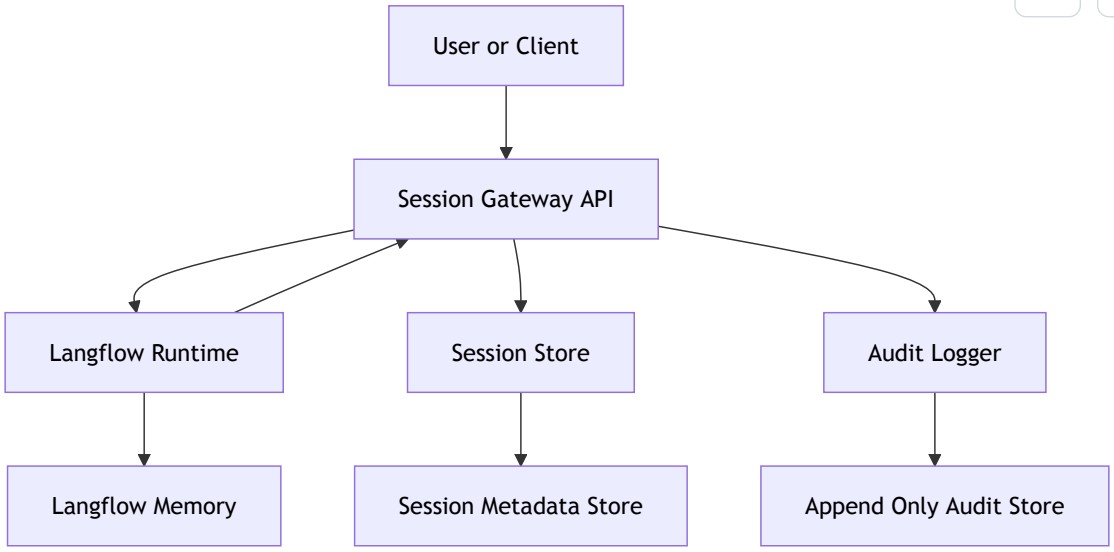
- Session Gateway becomes the system of record for sessions
- Langflow stays a deterministic executor of flows
- Session Gateway injects session id identity claims policy context and tool credentials

The goal is not to replace Langflow s session handling but to wrap and extend it so that

- Langflow continues to create and manage session id as it does today
- Session gateway augments this with lifecycle metadata and audit
- There is no fork or invasive change to Langflow

Key components

1 Session Gateway API
2 Session Store metadata state pointers
3 Audit Logger append only events



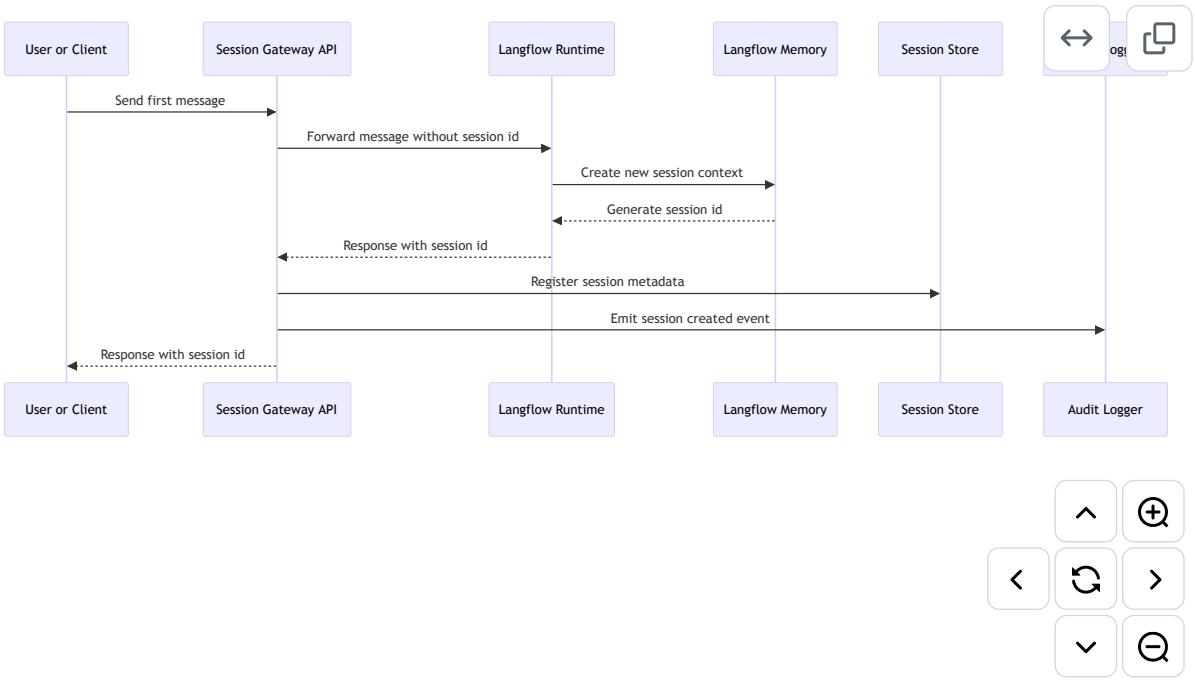
We can improve it later by adding

1 Policy Engine RBAC or PBAC
2 Concurrency Manager limits conflict detection

A session object should have following fields

Field	Example	Notes
session id	opaque uuid	non guessable
tenant id	acme prod	isolation key
user id	oidc sub	identity bound
flow id	flow 123	which flow
created at	timestamp	
expires at	timestamp	TTL enforced
status	active closed revoked	
region	eu west	residency
policy snapshot id	hash	what policies applied only if we add policy engine
retention profile	strict standard debug	
data flags	no store no train pii detected	

How this works step by step



1. Session creation

- User sends first message
- Session Gateway forwards request to Langflow
- Langflow generates session id internally
- Langflow returns session id in response

2. Session registration

- Session Gateway captures the session id
- Registers it in Session Store with
- tenant id
- user identity
- flow id
- creation time
- expiry policy
- Emits session created audit event

Langflow remains unaware of this metadata

- Subsequent messages include the same session_id
- Session Gateway
- validates session state
- forwards to Langflow unchanged
- Langflow
- uses session_id for memory lookup
- appends conversation state as usual

- Session Gateway can
- mark session expired or revoked
- stop forwarding requests for that session
- Langflow memory may still exist but
- session is logically closed
- no new execution allowed
-

Capability	Langflow	Session Gateway
User identity binding	No	Yes
Tenant isolation	Partial	Yes
Session expiry	No	Yes
Revocation	No	Yes
Audit trail	No	Yes

Minimal API surface

Implement this without changing Langflow internals too much

Session Gateway endpoints

```
POST sessions
POST sessions session id messages
POST sessions session id close
POST sessions session id revoke
```

conversations Service endpoints

```
GET conversations tenant id session id
GET conversations tenant id user id sessions
GET conversations tenant id user id session id
DELETE conversations tenant id session id
DELETE conversations tenant id user id sessions
DELETE conversations tenant id user id session id
GET conversations tenant id sessions?from ? to ?
DELETE conversations tenant id sessions?from ? to ?
```

Audit and observability

Event log is the system of record Emit structured events for

- session created closed revoked

- message received metadata only
- policy decision allow deny redact
- tool invoked tool result metadata
- conflict detected conflict resolved

Do not log payloads by default Make payload logging an explicit debug mode with short retention and redaction

Default policies

Retention profiles

- **strict:** store minimal metadata no transcript
- **standard:** transcript stored X days tool metadata Y days
- **debug:** payload sampling allowed TTL very short admin only

Data flags

- **no_train** always true in enterprise mode
- **no_store** optional per tenant or per session
- **pii_detected** triggers redaction or block